

Projet détaillé : Développement d'un système de supervision industrielle avec Raspberry Pi 4 sous Linux

Table des matières

1	Analyse fonctionnelle	3
1.1	Définition du besoin	3
1.2	Fonctions principales	3
1.3	Contraintes	4
1.4	Diagramme fonctionnel	4
1.5	Résumé de l'analyse fonctionnelle	5
2	Introduction aux manipulations d'un système Linux	5
2.1	Points de cours	5
2.1.1	Naviguer dans le système de fichiers	5
2.1.2	Gestion des fichiers et répertoires	6
2.1.3	Visualisation et édition de fichiers	7
2.1.4	Gestion des droits et des processus	8
2.1.5	Réseaux et connexions à distance	8
2.2	Questions et directives	9
2.3	Conclusion	10
3	Collecte de données en temps réel	10
3.1	Points de cours	11
3.1.1	Interfaçage des capteurs avec le Raspberry Pi	11
3.1.2	Programmation sous Linux pour l'acquisition de données	12
3.1.3	Protocoles de communication des capteurs	13
3.1.4	Exemple d'utilisation du bus I2C en Python	14
3.2	Questions et directives	15
3.3	Conclusion	16
4	Stockage et traitement des données	16
4.1	Points de cours	16
4.1.1	Choix de la base de données	16
4.1.2	Conception du schéma de la base de données	16
4.1.3	Traitement des données et détection d'anomalies	17
4.2	Questions et directives	17
5	Interface utilisateur	18
5.1	Option 1 : Interface web	18
5.1.1	Points de cours	18
5.1.2	Visualisation des données	18
5.1.3	Contrôle à distance des équipements	19

5.2	Option 2 : Interface graphique avec Tkinter	19
5.2.1	Points de cours	19
5.2.2	Visualisation des données	19
6	Intégration de ROS dans le projet	20
6.1	Points de cours	20
6.1.1	Présentation de ROS	20
6.1.2	Avantages de ROS	20
6.1.3	Installation de ROS sur Raspberry Pi	21
6.1.4	Création d'un workspace ROS	21
6.1.5	Développement des nœuds ROS	21
6.1.6	Intégration avec l'interface utilisateur	22
6.1.7	Gestion des services et actions	22
6.1.8	Outils utiles de ROS	23
6.2	Questions et directives	23
7	Communication à distance entre le Raspberry Pi et un PC	24
7.1	Points de cours	24
7.1.1	Accès distant via SSH	24
7.1.2	Transfert de fichiers avec SCP	25
7.1.3	Authentification par clés SSH	25
7.1.4	Sécurisation supplémentaire	26
7.1.5	Accès distant au bureau du Raspberry Pi	26
7.1.6	Transfert de fichiers avec SFTP	27
7.2	Questions et directives	27
7.3	Conclusion	28
8	Sécurité et communication	28
8.1	Points de cours	28
8.1.1	Sécurisation des communications avec SSL/TLS	28
8.1.2	Implémentation de l'authentification et du contrôle d'accès	29
8.2	Questions et directives	32
8.3	Conclusion	33
9	Tests et validation	33
9.1	Points de cours	33
9.1.1	Importance des tests	33
9.1.2	Types de tests	33
9.1.3	Tests unitaires en Python avec <code>unittest</code>	34
9.1.4	Autres frameworks de tests en Python	35
9.1.5	Tests d'intégration	36
9.1.6	Bonnes pratiques pour les tests	37
9.2	Questions et directives	38
9.3	Présentation des résultats des tests	38
9.4	Conclusion	38
10	Documentation et présentation	38
10.1	Instructions pour le rapport LaTeX professionnel	38

Introduction

Dans le cadre de ce projet, vous êtes amenés à développer un système de supervision industrielle en utilisant un Raspberry Pi 4 fonctionnant sous Linux. Ce système permettra de collecter des données en temps réel à partir de capteurs, de les stocker et de les traiter, puis de les visualiser via une interface utilisateur interactive. Vous aurez le choix entre développer une interface web ou une interface graphique locale en utilisant Tkinter. Vous mettrez également en place des fonctionnalités de contrôle à distance des équipements et assurerez la sécurité des communications.

Ce projet vise à vous familiariser avec les concepts clés de l'informatique industrielle, notamment l'interfaçage matériel, la programmation système, le développement d'interfaces utilisateur, les bases de données, les protocoles de sécurité, la communication à distance entre le Raspberry Pi et un PC, ainsi que l'utilisation de ROS pour une architecture modulaire et évolutive.

1 Analyse fonctionnelle

Avant de commencer le développement, il est essentiel de réaliser une analyse fonctionnelle détaillée pour définir précisément les besoins du système, les fonctions à réaliser, les interactions entre les différents modules, et les contraintes à respecter. Cette étape permettra de structurer votre approche et de garantir que toutes les exigences sont prises en compte.

1.1 Définition du besoin

Le système de supervision industrielle à développer doit permettre de :

- **Surveiller en temps réel** les paramètres industriels critiques tels que la température, la pression, l'humidité, etc.
- **Détecter des anomalies** ou des dépassements de seuils prédéfinis pour prévenir des situations dangereuses ou inefficaces.
- **Enregistrer les données** de manière continue pour une analyse ultérieure, permettant de tracer l'historique des mesures et d'identifier des tendances.
- **Fournir une interface utilisateur conviviale** permettant de visualiser les données en temps réel et l'historique, ainsi que d'interagir avec le système.
- **Permettre le contrôle à distance des équipements**, par exemple pour activer ou désactiver des dispositifs en fonction des besoins ou des alertes.
- **Assurer la sécurité des données et des communications**, en protégeant le système contre les accès non autorisés et en garantissant l'intégrité des données.
- **Utiliser une architecture logicielle modulaire et évolutive**, facilitée par l'intégration de ROS, pour permettre une maintenance aisée et des extensions futures.

1.2 Fonctions principales

Pour répondre aux besoins identifiés, le système doit réaliser les fonctions principales suivantes :

1. **Acquisition de données** : Lecture des capteurs, collecte des mesures, et prétraitement si nécessaire.
2. **Stockage des données** : Enregistrement sécurisé et structuré des mesures dans une base de données pour une consultation et une analyse ultérieures.

3. **Traitement des données** : Analyse en temps réel des données pour détecter des anomalies, générer des alertes, et extraire des informations pertinentes.
4. **Interface utilisateur** : Présentation des données de manière claire et intuitive, interaction avec l'utilisateur pour des commandes spécifiques ou des configurations.
5. **Contrôle des équipements** : Envoi de commandes pour agir sur les équipements en fonction des données collectées ou des interventions de l'utilisateur.
6. **Communication à distance** : Établissement d'une connexion sécurisée entre le Raspberry Pi et un PC ou d'autres dispositifs pour la supervision et le contrôle.
7. **Sécurité** : Mise en place de mécanismes de protection des données et des accès au système, y compris l'authentification, le chiffrement, et la gestion des permissions.
8. **Architecture modulaire avec ROS** : Utilisation de ROS pour assurer une communication efficace entre les différents modules du système, permettant une évolutivité et une maintenance facilitée.

1.3 Contraintes

Le système doit respecter les contraintes suivantes :

- **Matériel** : Utilisation du Raspberry Pi 4 et de capteurs compatibles, en tenant compte des limitations matérielles (performance, mémoire, interfaces disponibles).
- **Logiciel** : Utilisation du système d'exploitation Linux, programmation en Python, et utilisation de bibliothèques standards ou couramment utilisées dans l'industrie.
- **Temps réel** : Mise à jour régulière des données avec une latence minimale, assurant une réactivité suffisante pour la supervision en temps réel.
- **Sécurité** : Implémentation de protocoles sécurisés pour les communications et l'accès au système, respectant les bonnes pratiques en matière de cybersécurité.
- **Complexité** : Adapter le niveau de complexité pour qu'il soit réalisable dans le temps imparti, en priorisant les fonctionnalités essentielles.
- **Usabilité** : Conception d'une interface utilisateur intuitive et ergonomique, facilitant la prise en main et l'interaction avec le système.

1.4 Diagramme fonctionnel

Pour visualiser pleinement les fonctions du système et les interactions entre elles, il est important de réaliser un **diagramme fonctionnel**. Ce diagramme doit représenter les différents modules du système, les flux de données, les interfaces entre les composants, et les interactions avec l'utilisateur.

Éléments à inclure dans le diagramme :

- **Capteurs** : Dispositifs de mesure connectés au Raspberry Pi pour l'acquisition des données (température, pression, humidité, etc.).
- **Nœuds ROS** : Modules logiciels responsables de la collecte, du traitement, et du stockage des données, ainsi que du contrôle des équipements.
- **Base de données** : Système de stockage où sont enregistrées les mesures et les historiques.
- **Interface utilisateur** : Application web ou locale permettant la visualisation des données, la configuration du système, et l'envoi de commandes.
- **Équipements contrôlés** : Actionneurs ou dispositifs qui peuvent être activés ou désactivés par le système.
- **Réseau de communication** : Connexions (filaire ou sans fil) permettant la communication entre le Raspberry Pi, le PC distant, et éventuellement d'autres dispositifs.

- **Sécurité** : Mécanismes d’authentification, de chiffrement, et de gestion des permissions pour sécuriser le système.

Instructions pour la réalisation du diagramme :

- **Choisissez un outil de modélisation** : Vous pouvez utiliser des logiciels comme *draw.io*, *Lucidchart*, *Microsoft Visio*, ou tout autre outil de votre choix.
- **Identifiez les composants principaux** : Listez tous les modules fonctionnels du système, tels que les capteurs, les nœuds ROS, la base de données, l’interface utilisateur, etc.
- **Représentez les interactions** : Tracez les flux de données entre les composants, en indiquant clairement la direction des communications et le type de données échangées.
- **Utilisez des symboles standard** : Employez des symboles reconnus pour représenter les différents types de composants (par exemple, cylindres pour les bases de données, rectangles pour les processus, etc.).
- **Ajoutez des annotations** : Si nécessaire, ajoutez des notes explicatives pour clarifier certaines interactions ou spécificités du système.
- **Assurez la lisibilité** : Organisez le diagramme de manière claire et ordonnée, en évitant les croisements de lignes et en regroupant les éléments connexes.
- **Intégrez le diagramme dans le rapport** : Une fois le diagramme réalisé, insérez-le dans votre rapport LaTeX en utilisant le package `graphicx` :

```
\begin{figure}[h!]
    \centering
    \includegraphics[width=0.8\textwidth]{chemin/vers/votre/diagramme.png}
    \caption{Diagramme fonctionnel du système de supervision industrielle}
    \label{fig:diagramme_fonctionnel}
\end{figure}
```

1.5 Résumé de l’analyse fonctionnelle

L’analyse fonctionnelle a permis de définir clairement les objectifs du système, les fonctions à réaliser, et les contraintes à respecter. La réalisation d’un diagramme fonctionnel vous aidera à visualiser les interactions entre les différents modules, facilitant ainsi la phase de conception et de développement. Cette étape est cruciale pour assurer la cohérence, l’efficacité et la réussite de votre projet.

2 Introduction aux manipulations d’un système Linux

Avant de commencer le projet, il est essentiel de maîtriser les commandes de base sous Linux. Ces commandes vous permettront de naviguer dans le système de fichiers, de gérer les fichiers et répertoires, d’exécuter des programmes, et d’administrer le système. Cette section vous guidera à travers les commandes essentielles avec des exemples concrets pour renforcer votre compréhension.

2.1 Points de cours

2.1.1 Naviguer dans le système de fichiers

Le système de fichiers sous Linux est organisé sous forme d’une hiérarchie arborescente, avec le répertoire racine `/` à la base. Comprendre cette structure est fondamental pour accéder aux fichiers et répertoires nécessaires.

- **pwd (print working directory)** : Affiche le chemin complet du répertoire courant, vous permettant de savoir où vous vous trouvez dans l'arborescence.

```
1 $ pwd
2 /home/pi
```

Listing 1 – Utilisation de pwd

- **ls (list)** : Liste le contenu d'un répertoire. Il dispose de nombreuses options pour afficher des informations supplémentaires.

```
1 $ ls
2 Documents Downloads Music Pictures Videos
3
4 $ ls -l
5 total 20
6 drwxr-xr-x 2 pi pi 4096 Jan 1 10:00 Documents
7 drwxr-xr-x 2 pi pi 4096 Jan 1 10:00 Downloads
8 drwxr-xr-x 2 pi pi 4096 Jan 1 10:00 Music
9 drwxr-xr-x 2 pi pi 4096 Jan 1 10:00 Pictures
10 drwxr-xr-x 2 pi pi 4096 Jan 1 10:00 Videos
```

Listing 2 – Utilisation de ls

- **cd (change directory)** : Change le répertoire courant. Vous pouvez utiliser des chemins relatifs ou absolus.

```
1 $ cd /home/pi/Documents # Chemin absolu
2 $ pwd
3 /home/pi/Documents
4
5 $ cd ../Downloads # Chemin relatif
6 $ pwd
7 /home/pi/Downloads
8
9 $ cd ~ # Retour au r pertoire personnel
10 $ pwd
11 /home/pi
```

Listing 3 – Utilisation de cd

2.1.2 Gestion des fichiers et répertoires

La gestion efficace des fichiers et des répertoires est essentielle pour organiser votre travail et manipuler les données.

- **cp (copy)** : Copie des fichiers ou répertoires. L'option **-r** est nécessaire pour copier récursivement un répertoire.

```
1 $ cp fichier.txt copie_fichier.txt # Copie d'un fichier
2 $ cp -r dossier_original dossier_copie # Copie d'un
   r pertoire
```

Listing 4 – Utilisation de cp

- **mv (move)** : Déplace ou renomme des fichiers ou répertoires.

```
1 $ mv ancien_nom.txt nouveau_nom.txt # Renommer un fichier
2 $ mv fichier.txt /chemin/vers/dossier/ # D placer un fichier
```

Listing 5 – Utilisation de mv

- **rm (remove)** : Supprime des fichiers ou répertoires. Soyez prudent, car cette action est irréversible. L'option **-r** permet de supprimer récursivement un répertoire.

```
1 $ rm fichier.txt # Supprimer un fichier
2 $ rm -r dossier/ # Supprimer un
    r pertoire
```

Listing 6 – Utilisation de rm

- **mkdir (make directory)** : Crée un nouveau répertoire.

```
1 $ mkdir nouveau_dossier # Cr er un
    r pertoire
2 $ mkdir -p dossier1/dossier2 # Cr er des
    r pertoires imbriqu s
```

Listing 7 – Utilisation de mkdir

- **touch** : Crée un fichier vide ou met à jour la date de modification d'un fichier existant.

```
1 $ touch nouveau_fichier.txt # Cr er un fichier
    vide
2 $ touch fichier_existant.txt # Mettre jour la
    date du fichier
```

Listing 8 – Utilisation de touch

2.1.3 Visualisation et édition de fichiers

Consulter et modifier des fichiers sont des tâches courantes pour configurer le système ou écrire du code.

- **cat (concatenate)** : Affiche le contenu d'un fichier dans le terminal.

```
1 $ cat fichier.txt
2 Contenu du fichier s'affiche ici...
```

Listing 9 – Utilisation de cat

- **less** : Affiche le contenu d'un fichier page par page, pratique pour les fichiers volumineux.

```
1 $ less gros_fichier.log
```

Listing 10 – Utilisation de less

Naviguez avec les touches **flèche haut/bas** et quittez avec la touche **q**.

- **nano, vi, emacs** : Éditeurs de texte en ligne de commande pour modifier des fichiers.
- **nano** : Éditeur simple et facile à prendre en main.

```
1 $ nano fichier.txt
```

Listing 11 – Édition avec nano

- **vi** : Éditeur puissant avec une courbe d'apprentissage plus élevée.

```
1 $ vi fichier.txt
```

Listing 12 – Édition avec vi

- **emacs** : Éditeur extensible avec de nombreuses fonctionnalités.

```
1 $ emacs fichier.txt
```

Listing 13 – Édition avec emacs

2.1.4 Gestion des droits et des processus

Comprendre les permissions et la gestion des processus est crucial pour la sécurité et le bon fonctionnement du système.

- **chmod (change mode)** : Change les permissions d'un fichier ou répertoire. Les permissions déterminent qui peut lire (r), écrire (w), ou exécuter (x) un fichier.

```
1 $ ls -l script.sh
2 -rw-r--r-- 1 pi pi 0 Jan  1 12:00 script.sh
3
4 $ chmod +x script.sh           # Ajouter la permission d'
    ex cution
5 $ ls -l script.sh
6 -rwxr--r-- 1 pi pi 0 Jan  1 12:00 script.sh
7
8 $ chmod 755 script.sh         # D finir les permissions avec
    un code num rique
```

Listing 14 – Utilisation de chmod

- **ps (process status)** : Affiche les processus en cours d'exécution sur le système.

```
1 $ ps
2  PID TTY          TIME CMD
3 1234 pts/0        00:00:00 bash
4 5678 pts/0        00:00:00 ps
5
6 $ ps aux           # Afficher tous les
    processus
```

Listing 15 – Utilisation de ps

- **kill** : Termine un processus en lui envoyant un signal.

```
1 $ kill PID           # Remplacez PID par
    le num ro du processus
2
3 $ kill -9 PID        # Forcer l'arr t
    du processus
```

Listing 16 – Utilisation de kill

Vous pouvez trouver le PID d'un processus avec la commande **ps** ou **pgrep**.

2.1.5 Réseaux et connexions à distance

La gestion du réseau est essentielle pour la communication entre machines, surtout dans un contexte de supervision à distance.

- **ifconfig/ip** : Affiche ou configure les interfaces réseau.

```
1 $ ifconfig           # Afficher les
    interfaces r seau (ancienne m thode)
2
3 $ ip addr            # Afficher les
    adresses IP (nouvelle m thode)
```

Listing 17 – Utilisation de ifconfig

Exemple de sortie :

```

1 $ ip addr
2 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 ...
3     inet 127.0.0.1/8 scope host lo
4 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
5     inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0

```

— **ping** : Vérifie la connectivité avec une autre machine en envoyant des paquets ICMP.

```

1 $ ping 8.8.8.8 # Pinger le serveur
   DNS de Google
2 $ ping www.google.com # Pinger un nom de
   domaine

```

Listing 18 – Utilisation de ping

Interrompez le ping avec **Ctrl+C**.

— **ssh (Secure Shell)** : Se connecte à une machine distante en ligne de commande via le protocole SSH.

```

1 $ ssh pi@192.168.1.50 # Se connecter au
   Raspberry Pi l'adresse IP spécifiée

```

Listing 19 – Utilisation de ssh

— **scp (Secure Copy)** : Copie des fichiers vers ou depuis une machine distante de manière sécurisée.

```

1 $ scp fichier.txt pi@192.168.1.50:/home/pi/ # Copier un fichier
   vers le Raspberry Pi
2 $ scp pi@192.168.1.50:/home/pi/fichier.txt ./ # Copier un fichier
   depuis le Raspberry Pi

```

Listing 20 – Utilisation de scp

2.2 Questions et directives

Pour vous familiariser avec ces commandes, réalisez les exercices suivants :

1. Prise en main du terminal :

- Ouvrez un terminal sur le Raspberry Pi. Vous pouvez utiliser un écran connecté ou une connexion SSH depuis un autre ordinateur.
- Naviguez dans le système de fichiers en utilisant les commandes **pwd**, **ls**, et **cd**. Essayez de vous déplacer vers différents répertoires et observez les changements.

2. Gestion des fichiers :

- Créez un répertoire nommé **projet_supervision** dans votre répertoire personnel :

```

1 $ cd ~
2 $ mkdir projet_supervision

```

- À l'intérieur de ce répertoire, créez un fichier vide nommé **README.md** en utilisant la commande **touch** :

```

1 $ cd projet_supervision
2 $ touch README.md

```

- Copiez le fichier **README.md** en le renommant **INSTALL.md** :

```

1 $ cp README.md INSTALL.md

```

3. Édition de fichiers :

- Utilisez l'éditeur **nano** pour ajouter du contenu au fichier `README.md` :

```
1 $ nano README.md
```

Dans l'éditeur, tapez une description de votre projet, puis enregistrez avec **Ctrl+O** et quittez avec **Ctrl+X**.

- Affichez le contenu du fichier à l'aide de la commande **cat** :

```
1 $ cat README.md
```

Vous devriez voir le texte que vous avez entré.

4. Gestion des permissions :

- Modifiez les permissions du fichier `INSTALL.md` pour le rendre exécutable :

```
1 $ chmod +x INSTALL.md
```

- Vérifiez les permissions en utilisant la commande **ls -l** :

```
1 $ ls -l INSTALL.md
2 -rwxr-xr-x 1 pi pi 0 Jan  1 14:00 INSTALL.md
```

Notez le **x** qui indique que le fichier est exécutable.

5. Processus et tâches système :

- Affichez les processus en cours avec la commande **ps aux** :

```
1 $ ps aux
```

Cette commande affiche tous les processus avec des informations détaillées.

- Identifiez le PID de votre shell (par exemple, **bash**) et notez-le.

6. Réseaux :

- Affichez les paramètres réseau de votre Raspberry Pi avec **ifconfig** ou **ip addr** :

```
1 $ ip addr
```

Identifiez l'adresse IP associée à votre interface réseau (par exemple, `eth0` ou `wlan0`).

- Pinglez une adresse IP locale (par exemple, votre routeur) pour vérifier la connectivité :

```
1 $ ping 192.168.1.1
```

Si vous recevez des réponses, la connectivité est établie. Arrêtez le ping avec **Ctrl+C**.

2.3 Conclusion

La maîtrise des commandes de base sous Linux est indispensable pour le développement efficace sur le Raspberry Pi. En vous familiarisant avec ces outils, vous gagnerez en autonomie et en productivité, ce qui vous permettra de vous concentrer sur les aspects essentiels de votre projet de supervision industrielle.

3 Collecte de données en temps réel

La collecte de données en temps réel est une étape cruciale dans un système de supervision industrielle. Elle permet de surveiller les paramètres essentiels en continu, d'identifier rapidement les anomalies, et de prendre des décisions informées pour le contrôle des équipements. Cette section vous guidera à travers les concepts fondamentaux, les méthodes d'interfaçage des capteurs avec le Raspberry Pi, la programmation pour l'acquisition des données, et la compréhension des protocoles de communication.

3.1 Points de cours

3.1.1 Interfaçage des capteurs avec le Raspberry Pi

Le **Raspberry Pi** est une plateforme idéale pour la collecte de données grâce à ses multiples interfaces d'entrée/sortie et sa compatibilité avec divers protocoles de communication. Les interfaces couramment utilisées pour connecter des capteurs sont :

- **GPIO (General Purpose Input/Output)** : Broches polyvalentes permettant la communication numérique basique, configurables en entrée ou en sortie. Elles sont utilisées pour lire des signaux numériques (haut/bas) ou contrôler des dispositifs simples comme des LED ou des relais.
- **I2C (Inter-Integrated Circuit)** : Bus de communication série synchrone conçu pour la communication à courte distance entre des circuits intégrés. Il permet de connecter plusieurs périphériques à l'aide de seulement deux fils (SDA pour les données et SCL pour l'horloge). Chaque périphérique a une adresse unique, facilitant la communication avec plusieurs capteurs sur le même bus.
- **SPI (Serial Peripheral Interface)** : Bus de communication série synchrone, plus rapide que l'I2C. Il utilise quatre fils (MISO, MOSI, SCLK, CS) et est idéal pour les périphériques nécessitant des taux de transfert élevés, tels que les écrans ou certains capteurs rapides.
- **UART (Universal Asynchronous Receiver/Transmitter)** : Interface pour la communication série asynchrone, souvent utilisée pour les modules GPS, Bluetooth, ou des communications série RS-232.

Diagramme des broches du Raspberry Pi Pour une connexion réussie, il est important de connaître le schéma des broches (*pinout*) du Raspberry Pi. Vous pouvez vous référer au schéma officiel ou utiliser la commande suivante pour afficher le schéma dans le terminal :

```
1 $ pinout
```

Listing 21 – Afficher le schéma des broches

Exemple de connexion d'un capteur I2C Supposons que vous utilisez un capteur de température et de pression **BMP280** qui communique via I2C. Voici les étapes pour le connecter au Raspberry Pi :

1. Connexion des broches :

- Connectez la broche **VCC** du capteur au **3.3V** du Raspberry Pi.
- Connectez la broche **GND** du capteur au **GND** du Raspberry Pi.
- Connectez la broche **SDA** du capteur à la broche **GPIO 2** (SDA1) du Raspberry Pi.
- Connectez la broche **SCL** du capteur à la broche **GPIO 3** (SCL1) du Raspberry Pi.

2. Activation de l'interface I2C :

Utilisez l'outil `raspi-config` pour activer l'I2C :

```
1 $ sudo raspi-config
```

Listing 22 – Activation de l'I2C

Naviguez vers **Interfacing Options** → **I2C** et sélectionnez **Yes** pour activer.

3. Installation des outils I2C :

Installez les paquets nécessaires pour utiliser l'I2C :

```
1 $ sudo apt-get install -y python3-smbus i2c-tools
```

Listing 23 – Installation des outils I2C

4. Vérification de la connexion :

Utilisez la commande `i2cdetect` pour vérifier que le capteur est détecté sur le bus I2C :

```
1 $ i2cdetect -y 1
```

Listing 24 – Détection des périphériques I2C

Vous devriez voir l'adresse du capteur (par exemple, 0x76) apparaître dans la table.

3.1.2 Programmation sous Linux pour l'acquisition de données

La programmation pour l'acquisition de données implique d'interagir avec les interfaces matérielles du Raspberry Pi en utilisant des bibliothèques appropriées. Le langage **Python** est largement utilisé en raison de sa simplicité et de la disponibilité de nombreuses bibliothèques pour le matériel du Raspberry Pi.

Bibliothèques courantes

- **RPi.GPIO** : Bibliothèque standard pour contrôler les broches GPIO du Raspberry Pi. Elle permet de configurer les broches en entrée ou en sortie, de lire des signaux numériques, et de contrôler des périphériques simples.
- **smbus** : Fournit une interface pour communiquer via le bus I2C. Elle permet de lire et d'écrire des données sur les périphériques I2C.
- **spidev** : Permet la communication avec des périphériques SPI.
- **Adafruit_DHT** : Bibliothèque spécifique pour les capteurs DHT11 et DHT22 (température et humidité).

Exemple de lecture d'un capteur DHT22 (température et humidité) Le capteur **DHT22** est un capteur numérique capable de mesurer la température et l'humidité ambiante. Il se connecte au Raspberry Pi via une broche GPIO.

1. Connexion du capteur :

- Connectez la broche **VCC** du capteur au **3.3V** du Raspberry Pi.
- Connectez la broche **GND** du capteur au **GND** du Raspberry Pi.
- Connectez la broche **DATA** du capteur à une broche GPIO du Raspberry Pi (par exemple, **GPIO 4**).

2. Installation de la bibliothèque :

```
1 $ sudo pip3 install Adafruit_DHT
```

Listing 25 – Installation de la bibliothèque Adafruit_DHT

3. Script Python pour lire le capteur :

```
1 import Adafruit_DHT
2 import time
3
4 # Choisir le type de capteur
5 sensor = Adafruit_DHT.DHT22
6
7 # D finir le pin GPIO connect au capteur
8 pin = 4 # GPIO4
```

```

9
10 while True:
11     # Lire les donn es du capteur
12     humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
13
14     if humidity is not None and temperature is not None:
15         print('Temp rature={0:0.1f} C   Humidit ={1:0.1f}%'.
16               format(temperature, humidity))
17     else:
18         print(' chec  de lecture du capteur.')
19
20     time.sleep(2) # Attendre 2 secondes avant la prochaine
21                 lecture

```

Listing 26 – Lecture du capteur DHT22

4. Explication du code :

- `import Adafruit_DHT` : Importe la bibliothèque nécessaire pour communiquer avec le capteur.
- `sensor = Adafruit_DHT.DHT22` : Spécifie le modèle du capteur utilisé.
- `pin = 4` : Indique que le capteur est connecté à la broche GPIO 4.
- La boucle `while True` permet de lire les données en continu.
- `read_retry` tente de lire les données plusieurs fois en cas d'échec.
- Les valeurs de température et d'humidité sont affichées avec une précision d'une décimale.

Gestion des erreurs Il est important de gérer les exceptions et les erreurs de lecture pour assurer la robustesse du programme. Par exemple, vous pouvez utiliser des blocs `try...except` pour capturer les exceptions.

3.1.3 Protocoles de communication des capteurs

Comprendre les protocoles de communication est essentiel pour configurer correctement les échanges avec les capteurs et choisir les capteurs appropriés pour votre projet.

I2C (Inter-Integrated Circuit)

- **Caractéristiques** :
 - Bus série synchrone utilisant deux lignes : SDA (données) et SCL (horloge).
 - Permet la communication entre un maître (le Raspberry Pi) et plusieurs esclaves (les capteurs).
 - Chaque périphérique a une adresse unique sur le bus.
- **Avantages** :
 - Simple à mettre en œuvre avec peu de fils.
 - Idéal pour les capteurs à faible vitesse de données.
- **Limites** :
 - Vitesse de transfert limitée (jusqu'à 400 kbit/s en mode rapide).
 - Distance limitée entre les périphériques (quelques centimètres à quelques mètres).

SPI (Serial Peripheral Interface)

- **Caractéristiques** :

- Bus série synchrone utilisant quatre lignes principales : MOSI (Maître Out, Esclave In), MISO (Maître In, Esclave Out), SCLK (horloge), et CS (sélection du périphérique).
- Permet des communications à haute vitesse entre le maître et les esclaves.
- **Avantages :**
 - Vitesses de transfert élevées (plusieurs MHz).
 - Communication full-duplex (échange de données dans les deux sens simultanément).
- **Limites :**
 - Nécessite plus de fils que l'I2C.
 - Gestion plus complexe pour plusieurs périphériques (chaque périphérique nécessite une ligne CS dédiée).

One-Wire

- **Caractéristiques :**
 - Utilise une seule ligne pour la communication et l'alimentation des périphériques (en plus du GND).
 - Chaque périphérique a un identifiant unique de 64 bits.
- **Avantages :**
 - Réduction du câblage.
 - Idéal pour les réseaux de capteurs simples sur de longues distances.
- **Limites :**
 - Vitesse de transfert lente.
 - Complexité de la gestion des timings.

UART (Universal Asynchronous Receiver/Transmitter)

- **Caractéristiques :**
 - Communication série asynchrone sans signal d'horloge partagé.
 - Utilise des lignes TX (transmission) et RX (réception).
- **Avantages :**
 - Simple à mettre en œuvre.
 - Compatible avec de nombreux périphériques série comme les modules GPS, Bluetooth, etc.
- **Limites :**
 - Généralement limité à un seul périphérique par port UART.
 - Vitesse de communication inférieure aux bus synchrones.

3.1.4 Exemple d'utilisation du bus I2C en Python

Prenons l'exemple du capteur **BMP280** pour la mesure de la température et de la pression atmosphérique.

1. Installation des bibliothèques nécessaires :

```
1 $ sudo pip3 install Adafruit-BMP
```

Listing 27 – Installation de la bibliothèque BMP280

2. Script Python pour lire le capteur BMP280 :

```
1 import Adafruit_BMP.BMP280 as BMP280
2
3 # Cr er une instance du capteur
4 sensor = BMP280.BMP280()
```

```

5
6 # Lire la température
7 temperature = sensor.read_temperature()
8
9 # Lire la pression
10 pressure = sensor.read_pressure()
11
12 # Afficher les valeurs
13 print('Température = {0:0.2f} °C '.format(temperature))
14 print('Pression = {0:0.2f} Pa'.format(pressure))

```

Listing 28 – Lecture du capteur BMP280

3. Explication du code :

- `import Adafruit_BMP.BMP280 as BMP280` : Importe la classe BMP280 depuis la bibliothèque Adafruit.
- `sensor = BMP280.BMP280()` : Initialise le capteur avec les paramètres par défaut.
- `read_temperature()` et `read_pressure()` : Méthodes pour lire les valeurs actuelles du capteur.
- Les valeurs sont affichées avec une précision de deux décimales.

3.2 Questions et directives

Pour appliquer les connaissances acquises, réalisez les tâches suivantes :

1. Identification des capteurs :

- Sélectionnez au moins trois types de capteurs pertinents pour le projet, par exemple :
 - Capteur de température et d'humidité (DHT22)
 - Capteur de pression atmosphérique (BMP280)
 - Capteur de gaz ou de qualité de l'air (MQ-2)
- Justifiez votre choix en fonction des caractéristiques techniques, de la pertinence pour l'application industrielle, et de la facilité d'intégration avec le Raspberry Pi.

2. Connexion des capteurs :

- Pour chaque capteur, expliquez en détail comment le connecter physiquement au Raspberry Pi 4. Incluez :
 - Les schémas de connexion (vous pouvez les dessiner et les inclure dans le rapport).
 - Les broches utilisées sur le Raspberry Pi et sur le capteur.
 - Les éventuelles résistances de pull-up nécessaires (par exemple, pour le bus I2C ou les capteurs One-Wire).
 - Les précautions à prendre pour éviter d'endommager les composants (par exemple, ne pas appliquer 5V sur une entrée 3.3V).

3. Programmation de l'acquisition :

- Développez des scripts en Python pour lire les données de chaque capteur. Assurez-vous que chaque script peut s'exécuter indépendamment et afficher les mesures en temps réel.
- Incluez des commentaires dans le code pour expliquer les étapes importantes et faciliter la compréhension.
- Testez les scripts pour vérifier qu'ils fonctionnent correctement et qu'ils gèrent les éventuelles erreurs de lecture.

4. Lecture simultanée des capteurs :

- Modifiez vos scripts pour permettre la lecture simultanée des données de tous les capteurs. Vous pouvez utiliser :
 - Le **multithreading** : Créez un thread pour chaque capteur afin de les interroger en parallèle.
 - L'**asynchronisme** : Utilisez des bibliothèques comme **asyncio** pour gérer les lectures de manière asynchrone.
- Assurez-vous que les données sont synchronisées et que le programme reste stable.

5. Fréquence d'échantillonnage :

- Déterminez une fréquence d'échantillonnage appropriée pour chaque capteur en fonction de :
 - Les spécifications du capteur (temps de réponse, fréquence maximale recommandée).
 - Les besoins de l'application (rapidité de détection des changements, quantité de données générées).
- Expliquez vos choix et justifiez-les. Par exemple :
 - *Pour le DHT22, une lecture toutes les 2 secondes est appropriée car le capteur a un temps de réponse d'environ 2 secondes.*
 - *Pour le BMP280, une fréquence de 1 Hz (une lecture par seconde) est suffisante pour suivre les variations de pression atmosphérique.*

3.3 Conclusion

La collecte de données en temps réel sur le Raspberry Pi nécessite une compréhension approfondie des interfaces matérielles, des protocoles de communication, et des techniques de programmation adaptées. En maîtrisant ces compétences, vous serez en mesure de développer un système de supervision fiable et efficace, capable de surveiller les paramètres critiques de votre application industrielle.

N'oubliez pas de documenter soigneusement votre travail, de commenter votre code, et de tester chaque composant individuellement avant de les intégrer dans le système global.

4 Stockage et traitement des données

4.1 Points de cours

Le stockage des données permet de conserver un historique des mesures pour une analyse ultérieure, tandis que le traitement des données permet de détecter des anomalies et d'extraire des informations pertinentes.

4.1.1 Choix de la base de données

Deux types de bases de données peuvent être envisagés :

- **Bases de données relationnelles (SQL)** : Comme SQLite, MySQL, ou PostgreSQL.
- **Bases de données non relationnelles (NoSQL)** : Comme MongoDB.

4.1.2 Conception du schéma de la base de données

Une bonne conception du schéma facilite les opérations de stockage et de requête.

- **Tables nécessaires** : Une table principale pour les mesures avec des champs tels que *id*, *capteur*, *type_mesure*, *valeur*, *unite*, *horodatage*.
- **Indexation** : Créer des index sur les champs fréquemment utilisés.

— **Intégrité des données** : Définir des contraintes pour assurer la cohérence des données.
Exemple de création d'une base de données SQLite en Python :

```
1 import sqlite3
2
3 # Connexion à la base de données
4 conn = sqlite3.connect('mesures.db')
5
6 # Création d'un curseur
7 cursor = conn.cursor()
8
9 # Création de la table des mesures
10 cursor.execute('''
11 CREATE TABLE IF NOT EXISTS mesures (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     capteur TEXT NOT NULL,
14     type_mesure TEXT NOT NULL,
15     valeur REAL NOT NULL,
16     unite TEXT NOT NULL,
17     horodatage DATETIME DEFAULT CURRENT_TIMESTAMP
18 )
19 ''')
20
21 # Validation des changements
22 conn.commit()
23
24 # Fermeture de la connexion
25 conn.close()
```

Listing 29 – Création d'une base de données SQLite

4.1.3 Traitement des données et détection d'anomalies

Analyser les données permet de détecter des conditions anormales ou critiques.

- **Seuils fixes** : Définir des valeurs limites pour chaque type de mesure.
- **Analyse statistique** : Calculer des statistiques pour détecter des tendances.
- **Algorithmes avancés** : Utiliser des techniques d'apprentissage automatique.

Exemple de détection d'anomalies :

```
1 SEUIL_TEMPERATURE_MAX = 75.0 # degrés Celsius
2 SEUIL_TEMPERATURE_MIN = 0.0
3
4 if temperature > SEUIL_TEMPERATURE_MAX or temperature <
   SEUIL_TEMPERATURE_MIN:
5     print('Alerte : Température hors limites !')
6     # Actions supplémentaires
```

Listing 30 – Détection de dépassement de seuil

4.2 Questions et directives

1. Choix de la base de données :

- Sélectionnez une base de données adaptée à votre projet.
- Justifiez votre choix.

2. **Conception du schéma :**
 - Dessinez le schéma de votre base de données.
 - Détaillez les tables, les champs, les types de données.
3. **Implémentation :**
 - Créez la base de données.
 - Implémentez les requêtes nécessaires pour insérer et récupérer les données.
4. **Détection d'anomalies :**
 - Définissez des seuils pour chaque type de mesure.
 - Implémentez un script qui analyse les données en temps réel.
5. **Visualisation des données historiques :**
 - Préparez des requêtes permettant d'afficher les données sur des périodes spécifiques.

5 Interface utilisateur

Vous avez le choix entre deux options pour développer l'interface utilisateur : une interface web en utilisant Flask ou Django, ou une interface graphique locale en utilisant Tkinter.

5.1 Option 1 : Interface web

5.1.1 Points de cours

Développer une interface web permet d'accéder au système depuis n'importe quel navigateur, offrant ainsi une grande flexibilité.

- **Flask** : Un micro-framework web léger pour Python.
- **Django** : Un framework web complet pour Python.
- **Choix du framework** : Flask pour des applications simples, Django pour des projets plus complexes.

Exemple de route Flask pour afficher les données :

```

1 from flask import Flask, render_template
2 import sqlite3
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8     # Connexion à la base de données
9     # Récupération des données
10    # Rendu du template avec les données
11    pass
12
13 if __name__ == '__main__':
14     app.run(debug=True)
```

Listing 31 – Route Flask pour afficher les mesures

5.1.2 Visualisation des données

L'utilisation de bibliothèques JavaScript permet de créer des graphiques interactifs.

- **Chart.js** : Bibliothèque JavaScript pour créer des graphiques.
- **D3.js** : Bibliothèque pour manipuler les documents basés sur des données.

— Mise à jour en temps réel : Utilisation d’AJAX ou de WebSockets.

Exemple d’utilisation de Chart.js dans le template HTML :

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Supervision Industrielle</title>
5     <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
6 </head>
7 <body>
8     <!-- Contenu du graphique -->
9 </body>
10 </html>
```

Listing 32 – Intégration de Chart.js

5.1.3 Contrôle à distance des équipements

- **Actions à distance** : Ajouter des boutons ou des contrôles sur l’interface web permettant à l’utilisateur d’envoyer des commandes au Raspberry Pi pour contrôler les équipements.
- **Sécurité** : Mettre en place une authentification pour restreindre l’accès aux fonctions de contrôle.

5.2 Option 2 : Interface graphique avec Tkinter

5.2.1 Points de cours

Tkinter est la bibliothèque standard de Python pour créer des interfaces graphiques.

- **Création de fenêtres** : Classe **Tk** pour la fenêtre principale.
- **Widgets de base** : Labels, boutons, champs de saisie, etc.
- **Disposition des widgets** : Gestionnaires de placement **pack**, **grid**, **place**.
- **Événements et callbacks** : Gestion des événements pour l’interactivité.
- **Mise à jour en temps réel** : Méthode **after** pour exécuter périodiquement une fonction.

Exemple de création d’une fenêtre Tkinter :

```
1 import tkinter as tk
2
3 root = tk.Tk()
4 root.title("Supervision Industrielle")
5
6 # Cr ation de widgets
7
8 root.mainloop()
```

Listing 33 – Fenêtre Tkinter de base

5.2.2 Visualisation des données

Pour afficher des graphiques, on peut utiliser **matplotlib** avec **FigureCanvasTkAgg**.

Exemple d’intégration de matplotlib avec Tkinter :

```
1 import tkinter as tk
2 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

```

3 import matplotlib.pyplot as plt
4
5 root = tk.Tk()
6 root.title("Supervision Industrielle")
7
8 fig, ax = plt.subplots()
9 canvas = FigureCanvasTkAgg(fig, master=root)
10 canvas.get_tk_widget().pack()
11
12 # Mise      jour du graphique
13
14 root.mainloop()

```

Listing 34 – Graphique en temps réel avec matplotlib

6 Intégration de ROS dans le projet

6.1 Points de cours

L'utilisation de ROS (Robot Operating System) peut enrichir votre projet en apportant une architecture modulaire et extensible pour la gestion des capteurs, le traitement des données, et le contrôle des équipements.

6.1.1 Présentation de ROS

ROS est un framework logiciel open-source conçu pour simplifier la création de systèmes robotiques complexes. Bien que principalement utilisé en robotique, ROS est également adapté pour d'autres domaines nécessitant une communication efficace entre différents processus ou modules.

Principes de base de ROS :

- **Nœuds** : Programmes exécutables qui effectuent des calculs. Chaque nœud est conçu pour être simple et réaliser une tâche spécifique.
- **Messages** : Structures de données utilisées pour la communication entre nœuds.
- **Topics** : Canaux de communication sur lesquels les nœuds peuvent publier ou souscrire des messages.
- **Services** : Permettent une communication synchrone entre nœuds via des requêtes et des réponses.
- **Master ROS** : Processus central qui gère l'enregistrement des nœuds et facilite leur communication.

6.1.2 Avantages de ROS

- **Modularité** : Facilite le développement et la maintenance en découpant le système en composants indépendants.
- **Scalabilité** : Permet d'ajouter facilement de nouveaux capteurs ou fonctionnalités.
- **Standardisation** : Utilisé largement dans la recherche et l'industrie, favorisant la collaboration.
- **Large écosystème** : Dispose de nombreuses bibliothèques et outils pour divers besoins.

6.1.3 Installation de ROS sur Raspberry Pi

Pour un Raspberry Pi 4, il est recommandé d'installer **ROS Noetic**, compatible avec Python 3 et les versions récentes de Raspbian.

Étapes d'installation :

1. Mise à jour du système :

```
1 $ sudo apt update
2 $ sudo apt upgrade
```

2. Ajout des sources ROS :

```
1 $ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu buster
    main" > /etc/apt/sources.list.d/ros-latest.list'
```

3. Ajout de la clé du dépôt :

```
1 $ sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --
    recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

4. Installation de ROS :

```
1 $ sudo apt update
2 $ sudo apt install ros-noetic-ros-base
```

5. Initialisation de rosdep :

```
1 $ sudo rosdep init
2 $ rosdep update
```

6. Configuration de l'environnement ROS :

```
1 $ echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
2 $ source ~/.bashrc
```

6.1.4 Création d'un workspace ROS

Le workspace est l'endroit où vous développerez vos packages ROS.

```
1 $ mkdir -p ~/ros_workspace/src
2 $ cd ~/ros_workspace/
3 $ catkin_make
4 $ echo "source ~/ros_workspace/devel/setup.bash" >> ~/.bashrc
5 $ source ~/.bashrc
```

6.1.5 Développement des nœuds ROS

Exemple de nœud pour un capteur de température :

1. Création du package :

```
1 $ cd ~/ros_workspace/src
2 $ catkin_create_pkg capteurs rospy std_msgs
3 $ cd ..
4 $ catkin_make
```

2. Écriture du script du nœud :

Créez un fichier nommé *temperature_sensor_node.py* dans le dossier *capteurs/scripts*.

```

1  #!/usr/bin/env python3
2  import rospy
3  from std_msgs.msg import Float32
4  import Adafruit_DHT
5
6  def temperature_publisher():
7      pub = rospy.Publisher('temperature', Float32, queue_size=10)
8      rospy.init_node('temperature_sensor_node', anonymous=True)
9      rate = rospy.Rate(1) # 1 Hz
10     sensor = Adafruit_DHT.DHT22
11     pin = 4 # GPIO4
12
13     while not rospy.is_shutdown():
14         humidity, temperature = Adafruit_DHT.read_retry(sensor,
15             pin)
16         if temperature is not None:
17             rospy.loginfo(f"Temp rature : {temperature:.1f} C ")
18             pub.publish(temperature)
19         else:
20             rospy.logwarn(" chec de lecture du capteur.")
21             rate.sleep()
22
23 if __name__ == '__main__':
24     try:
25         temperature_publisher()
26     except rospy.ROSInterruptException:
27         pass

```

Listing 35 – Nœud ROS pour le capteur de température

3. Rendre le script exécutable :

```
1 $ chmod +x temperature_sensor_node.py
```

4. Mise à jour du fichier CMakeLists.txt :

Ajoutez le script dans la section des scripts exécutables.

5. Compilation du workspace :

```
1 $ cd ~/ros_workspace/
2 $ catkin_make
```

6.1.6 Intégration avec l'interface utilisateur

— Interface web avec Flask :

Vous pouvez utiliser **rosbridge_suite** pour permettre la communication entre ROS et une interface web via WebSockets. Cela permet à votre application web de souscrire aux topics ROS et de recevoir les données en temps réel.

— Interface graphique avec Tkinter :

Votre application Tkinter peut être un nœud ROS qui souscrit aux topics des capteurs. Vous pouvez utiliser les callbacks ROS pour mettre à jour l'interface utilisateur avec les nouvelles données.

6.1.7 Gestion des services et actions

Exemple de service pour contrôler un actionneur :

```

1  #!/usr/bin/env python3
2  import rospy
3  from std_srvs.srv import SetBool, SetBoolResponse
4  import RPi.GPIO as GPIO
5
6  GPIO.setmode(GPIO.BCM)
7  GPIO.setup(18, GPIO.OUT)
8
9  def handle_control_request(req):
10     if req.data:
11         GPIO.output(18, GPIO.HIGH)
12         message = "Actionneur activ "
13     else:
14         GPIO.output(18, GPIO.LOW)
15         message = "Actionneur d sactiv "
16     return SetBoolResponse(success=True, message=message)
17
18 def control_service():
19     rospy.init_node('control_service_node')
20     service = rospy.Service('control_actionneur', SetBool,
21                             handle_control_request)
22     rospy.loginfo("Service de contr le pr t.")
23     rospy.spin()
24
25 if __name__ == '__main__':
26     control_service()

```

Listing 36 – Nœud ROS pour le contrôle d’un actionneur

6.1.8 Outils utiles de ROS

- **roslaunch** : Permet de lancer plusieurs nœuds ROS à partir d’un fichier de configuration.
- **rqt** : Ensemble d’outils graphiques pour visualiser les données, les graphiques, et l’état du système.
- **rosbag** : Enregistre et lit les messages publiés sur les topics, utile pour l’analyse hors ligne.

6.2 Questions et directives

1. **Installation de ROS sur le Raspberry Pi** :
 - Suivez les étapes pour installer ROS Noetic.
 - Vérifiez que l’installation est réussie en exécutant des commandes ROS de base.
2. **Développement des nœuds ROS** :
 - Créez des nœuds pour chaque capteur sélectionné.
 - Assurez-vous que les nœuds publient les données sur des topics appropriés.
3. **Intégration avec l’interface utilisateur** :
 - Si vous utilisez une interface web, intégrez **rosbridge_suite** pour communiquer avec ROS.
 - Si vous utilisez Tkinter, faites en sorte que votre application soit un nœud ROS.
4. **Gestion des services** :
 - Implémentez des services pour le contrôle des équipements.
 - Testez les services en appelant des requêtes depuis des scripts clients.

5. Tests et validation :

- Testez la communication entre les nœuds ROS.
- Vérifiez que les données sont correctement transmises et reçues.

7 Communication à distance entre le Raspberry Pi et un PC

La communication à distance est une composante essentielle pour gérer efficacement votre Raspberry Pi depuis un autre ordinateur. Elle permet de faciliter le développement, la maintenance, et le déploiement de votre projet sans avoir besoin d'un accès physique direct au Raspberry Pi. Dans cette section, nous allons explorer les méthodes pour établir une connexion sécurisée entre le Raspberry Pi et un PC, en utilisant des outils tels que SSH et SCP, ainsi que la gestion des clés SSH pour renforcer la sécurité.

7.1 Points de cours

7.1.1 Accès distant via SSH

Le protocole **SSH** (Secure Shell) est un protocole de communication sécurisé qui permet de se connecter à une machine distante via le réseau. Il offre un chiffrement des données échangées, assurant la confidentialité et l'intégrité des informations transmises.

Activation du serveur SSH sur le Raspberry Pi Par défaut, le serveur SSH n'est pas toujours activé sur le Raspberry Pi pour des raisons de sécurité. Pour l'activer, vous pouvez utiliser l'outil de configuration `raspi-config` :

```
1 $ sudo raspi-config
```

Listing 37 – Activation du serveur SSH

Naviguez dans le menu :

1. Sélectionnez **Interfacing Options**.
2. Choisissez **SSH**.
3. Sélectionnez **Yes** pour activer le serveur SSH.
4. Quittez `raspi-config` en enregistrant les modifications.

Vérification du statut du serveur SSH Vous pouvez vérifier si le serveur SSH est actif en utilisant la commande :

```
1 $ sudo systemctl status ssh
```

Listing 38 – Vérification du statut du service SSH

Connexion depuis le PC Une fois le serveur SSH activé sur le Raspberry Pi, vous pouvez vous y connecter depuis un PC (sous Linux, macOS ou Windows avec un client SSH comme PuTTY) en utilisant la commande `ssh` :

```
1 $ ssh pi@192.168.1.10
```

Listing 39 – Connexion SSH depuis le PC

- `pi` est le nom d'utilisateur sur le Raspberry Pi (par défaut `pi`).
- `192.168.1.10` est l'adresse IP du Raspberry Pi sur le réseau local.

Lors de la première connexion, vous serez invité à accepter la clé d'authentification du serveur :

```
1 The authenticity of host '192.168.1.10 (192.168.1.10)' can't be
  established.
2 ECDSA key fingerprint is SHA256:...
3 Are you sure you want to continue connecting (yes/no)? yes
```

Tapez **yes** pour accepter. Vous devrez ensuite entrer le mot de passe de l'utilisateur **pi**.

7.1.2 Transfert de fichiers avec SCP

Le protocole **SCP** (Secure Copy Protocol) permet de copier des fichiers entre un hôte local et un hôte distant de manière sécurisée, en utilisant le protocole SSH.

```
1 $ scp fichier.txt pi@192.168.1.10:/home/pi/
```

Listing 40 – Copie d'un fichier vers le Raspberry Pi

- **fichier.txt** est le fichier sur votre PC que vous souhaitez copier.
- **pi@192.168.1.10:/home/pi/** est la destination sur le Raspberry Pi.

```
1 $ scp pi@192.168.1.10:/home/pi/fichier.txt /chemin/vers/dossier/local/
```

Listing 41 – Copie d'un fichier depuis le Raspberry Pi

- **/chemin/vers/dossier/local/** est le répertoire sur votre PC où vous souhaitez copier le fichier.

Copier un répertoire entier Pour copier un répertoire et son contenu, utilisez l'option **-r** (récursif) :

```
1 $ scp -r dossier/ pi@192.168.1.10:/home/pi/
```

Listing 42 – Copie d'un répertoire vers le Raspberry Pi

7.1.3 Authentification par clés SSH

L'authentification par clés SSH permet de renforcer la sécurité et de faciliter la connexion sans avoir à saisir le mot de passe à chaque fois.

Génération d'une paire de clés SSH sur le PC Exécutez la commande suivante sur votre PC pour générer une paire de clés RSA de 4096 bits :

```
1 $ ssh-keygen -t rsa -b 4096 -C "votre_email@example.com"
```

Listing 43 – Génération d'une paire de clés SSH

- **-t rsa** spécifie le type de clé (RSA).
- **-b 4096** définit la taille de la clé en bits.
- **-C "votre_email@example.com"** ajoute un commentaire (généralement votre email).

Vous serez invité à choisir un emplacement pour stocker les clés (appuyez sur **Entrée** pour accepter le chemin par défaut **~/.ssh/id_rsa**) et à saisir une passphrase facultative pour protéger votre clé privée.

Copie de la clé publique sur le Raspberry Pi Pour autoriser votre PC à se connecter au Raspberry Pi sans mot de passe, vous devez copier la clé publique sur le Raspberry Pi :

```
1 $ ssh-copy-id pi@192.168.1.10
```

Listing 44 – Copie de la clé publique sur le Raspberry Pi

Cette commande va ajouter la clé publique (`id_rsa.pub`) à la liste des clés autorisées (`authorized_keys`) sur le Raspberry Pi.

Connexion sans mot de passe Désormais, vous pouvez vous connecter au Raspberry Pi sans saisir le mot de passe :

```
1 $ ssh pi@192.168.1.10
```

Listing 45 – Connexion SSH sans mot de passe

Si vous aviez défini une passphrase lors de la génération de la clé, vous devrez la saisir pour déverrouiller la clé privée.

7.1.4 Sécurisation supplémentaire

Pour renforcer la sécurité de votre système, vous pouvez :

- **Désactiver l’authentification par mot de passe** : Une fois que l’authentification par clés est configurée, vous pouvez désactiver l’authentification par mot de passe sur le Raspberry Pi pour empêcher les tentatives de connexion non autorisées.

```
1 $ sudo nano /etc/ssh/sshd_config
```

Listing 46 – Édition du fichier de configuration SSH

Dans le fichier, modifiez les lignes suivantes :

```
1 # Changez ces lignes
2 PasswordAuthentication no
3 PermitRootLogin no
```

Enregistrez les modifications (`Ctrl+O` puis `Entrée`) et quittez l’éditeur (`Ctrl+X`). Redémarrez le service SSH :

```
1 $ sudo systemctl restart ssh
```

Listing 47 – Redémarrage du service SSH

- **Changer le port SSH** : Modifier le port par défaut (22) pour réduire les risques d’attaques automatisées.
- **Utiliser un pare-feu** : Configurez un pare-feu (par exemple, `ufw` ou `iptables`) pour contrôler les connexions entrantes.
- **Surveiller les tentatives de connexion** : Utilisez des outils comme `fail2ban` pour bloquer les adresses IP après plusieurs tentatives de connexion échouées.

7.1.5 Accès distant au bureau du Raspberry Pi

Si vous avez besoin d’accéder à l’interface graphique du Raspberry Pi, vous pouvez utiliser le protocole **VNC** (Virtual Network Computing).

Activation du serveur VNC Activez le serveur VNC sur le Raspberry Pi via `raspi-config` :

```
1 $ sudo raspi-config
```

Listing 48 – Activation du serveur VNC

Naviguez dans `Interfacing Options` → `VNC` et sélectionnez **Yes**.

Connexion depuis le PC Téléchargez et installez un client VNC sur votre PC (par exemple, RealVNC Viewer). Connectez-vous en entrant l'adresse IP du Raspberry Pi :

1 192.168.1.10

Listing 49 – Adresse pour la connexion VNC

Vous serez invité à saisir le nom d'utilisateur et le mot de passe du Raspberry Pi.

7.1.6 Transfert de fichiers avec SFTP

En plus de SCP, vous pouvez utiliser **SFTP** (SSH File Transfer Protocol) pour transférer des fichiers de manière sécurisée. Des clients SFTP graphiques comme FileZilla ou WinSCP facilitent le transfert de fichiers.

Utilisation de FileZilla

1. Téléchargez et installez FileZilla sur votre PC.
2. Ouvrez FileZilla et saisissez les informations de connexion :
 - **Hôte** : `sftp://192.168.1.10`
 - **Nom d'utilisateur** : `pi`
 - **Mot de passe** : votre mot de passe sur le Raspberry Pi
 - **Port** : 22
3. Cliquez sur **Connexion rapide**.

Vous pourrez alors naviguer dans les répertoires du Raspberry Pi et transférer des fichiers par glisser-déposer.

7.2 Questions et directives

Pour mettre en pratique les concepts abordés, réalisez les activités suivantes :

1. **Configurer l'accès SSH** :
 - Activez le serveur SSH sur votre Raspberry Pi en utilisant `raspi-config`.
 - Vérifiez que le serveur SSH est en cours d'exécution.
2. **Se connecter au Raspberry Pi depuis le PC** :
 - Identifiez l'adresse IP du Raspberry Pi en utilisant `ifconfig` ou `hostname -I`.
 - Depuis votre PC, établissez une connexion SSH avec le Raspberry Pi.
 - Explorez le système de fichiers du Raspberry Pi à partir du terminal du PC.
3. **Configurer l'authentification par clés SSH** :
 - Générer une paire de clés SSH sur votre PC.
 - Copier la clé publique sur le Raspberry Pi en utilisant `ssh-copy-id`.
 - Tester la connexion SSH sans mot de passe.
4. **Sécuriser le serveur SSH** :
 - Désactiver l'authentification par mot de passe sur le Raspberry Pi.
 - Configurer un pare-feu pour autoriser uniquement les connexions SSH depuis des adresses IP spécifiques (facultatif).
5. **Transférer des fichiers** :
 - Utiliser `scp` pour copier des fichiers entre le PC et le Raspberry Pi.
 - Utiliser un client SFTP graphique pour gérer les fichiers de manière plus conviviale.
6. **Accéder au bureau du Raspberry Pi (facultatif)** :
 - Activer le serveur VNC sur le Raspberry Pi.
 - Se connecter au Raspberry Pi depuis le PC en utilisant un client VNC.

7.3 Conclusion

La maîtrise des techniques de communication à distance est essentielle pour le développement et la maintenance efficaces de votre projet de supervision industrielle. En utilisant SSH et SCP, vous pouvez gérer votre Raspberry Pi de manière sécurisée, transférer des fichiers, et exécuter des commandes à distance. L'authentification par clés SSH renforce la sécurité de votre système tout en simplifiant le processus de connexion. Ces compétences vous permettront de travailler de manière plus flexible et efficace, que ce soit en laboratoire ou sur le terrain.

8 Sécurité et communication

Dans un système de supervision industrielle, la sécurité est un enjeu primordial. Il est essentiel de protéger le système contre les accès non autorisés, d'assurer la confidentialité et l'intégrité des données, et de garantir que seules les personnes autorisées peuvent interagir avec le système. Cette section aborde les concepts clés de la sécurité des communications, notamment la mise en place de SSL/TLS pour chiffrer les échanges, et l'implémentation d'un système d'authentification et de contrôle d'accès.

8.1 Points de cours

8.1.1 Sécurisation des communications avec SSL/TLS

Les protocoles **SSL** (Secure Sockets Layer) et **TLS** (Transport Layer Security) sont des protocoles cryptographiques qui fournissent une communication sécurisée sur un réseau informatique. Ils sont largement utilisés pour sécuriser les communications sur Internet, notamment pour les transactions sensibles telles que les paiements en ligne.

Pourquoi utiliser SSL/TLS ?

- **Confidentialité** : Le chiffrement des données empêche les tiers non autorisés de lire les informations échangées.
- **Intégrité** : Les mécanismes d'authentification garantissent que les données n'ont pas été modifiées en transit.
- **Authentification** : Les certificats SSL/TLS permettent de vérifier l'identité du serveur, évitant les attaques de type "man-in-the-middle".

Certificats SSL/TLS Un certificat SSL/TLS est un fichier de données qui associe une clé cryptographique aux informations d'une organisation. Lorsqu'il est installé sur un serveur, il active le protocole HTTPS (HTTP sécurisé) et permet des connexions sécurisées entre le serveur web et le navigateur.

Génération d'un certificat SSL auto-signé Pour les environnements de développement ou les applications internes, il est courant d'utiliser un certificat auto-signé. Bien qu'ils ne soient pas validés par une autorité de certification (CA), ils permettent de chiffrer les communications.

```
1 $ openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout server.  
key -out server.crt
```

Listing 50 – Génération d'un certificat SSL auto-signé

Explication des options :

- **openssl** : Commande pour utiliser la boîte à outils OpenSSL.

- **req** : Utilitaire pour créer et gérer des demandes de signature de certificat (CSR) et des certificats.
- **-x509** : Génère un certificat auto-signé plutôt qu’une demande de signature de certificat.
- **-nodes** : Pas de chiffrement du fichier de clé privée (pas de mot de passe).
- **-days 365** : Validité du certificat en jours.
- **-newkey rsa:2048** : Crée une nouvelle clé privée RSA de 2048 bits.
- **-keyout server.key** : Fichier de sortie pour la clé privée.
- **-out server.crt** : Fichier de sortie pour le certificat.

Lors de l’exécution de cette commande, vous serez invité à entrer des informations pour le certificat, telles que le pays, l’État, l’organisation, le nom commun (CN), etc. Le nom commun doit correspondre au nom de domaine ou à l’adresse IP du serveur.

Configuration de votre application Flask pour utiliser SSL/TLS Pour servir votre application Flask via HTTPS, vous pouvez spécifier les certificats lors du lancement de l’application :

```
1 if __name__ == '__main__':
2     app.run(debug=True, host='0.0.0.0', ssl_context=('server.crt',
    server.key'))
```

Listing 51 – Lancement de Flask avec SSL

Remarque : L’utilisation de SSL/TLS directement avec le serveur de développement intégré de Flask est acceptable pour des environnements de développement ou de test, mais pour un déploiement en production, il est recommandé d’utiliser un serveur web dédié comme **Nginx** ou **Apache** pour gérer SSL/TLS.

8.1.2 Implémentation de l’authentification et du contrôle d’accès

Pour protéger les fonctionnalités sensibles de votre application, telles que le contrôle à distance des équipements, il est essentiel d’implémenter un système d’authentification et de gestion des utilisateurs.

Gestion des utilisateurs La gestion des utilisateurs implique :

- **Enregistrement** : Permettre aux utilisateurs de créer un compte avec un nom d’utilisateur et un mot de passe.
- **Authentification** : Vérifier l’identité de l’utilisateur lors de la connexion.
- **Gestion des sessions** : Maintenir l’état de connexion de l’utilisateur entre les requêtes.
- **Réinitialisation du mot de passe** : Fournir une méthode pour réinitialiser le mot de passe en cas d’oubli.

Contrôle d’accès et autorisations Le contrôle d’accès consiste à définir des rôles et des permissions pour déterminer quelles ressources ou fonctionnalités chaque utilisateur peut accéder.

- **Rôles** : Par exemple, **admin**, **opérateur**, **observateur**.
- **Permissions** : Actions spécifiques que les utilisateurs peuvent effectuer, comme **visualiser les données**, **contrôler les équipements**, **gérer les utilisateurs**.

Utilisation de Flask-Login pour gérer les utilisateurs **Flask-Login** est une extension Flask qui facilite la gestion des sessions utilisateur. Elle gère l’état de connexion, la redirection des utilisateurs non authentifiés, et fournit un ensemble de fonctionnalités pour simplifier l’authentification.

1. Installation de Flask-Login

```
1 $ sudo pip3 install flask-login
```

Listing 52 – Installation de Flask-Login

2. Configuration de Flask-Login

```
1 from flask import Flask
2 from flask_login import LoginManager, UserMixin, login_user,
   logout_user, login_required, current_user
3
4 app = Flask(__name__)
5 app.secret_key = 'votre_cl _secre te' # N ecessaire pour les
   sessions
6
7 login_manager = LoginManager()
8 login_manager.init_app(app)
9 login_manager.login_view = 'login' # Nom de la vue pour la
   page de connexion
```

Listing 53 – Configuration de Flask-Login

3. Création du modèle utilisateur

Vous devez créer une classe utilisateur qui hérite de `UserMixin` et implémente au minimum les propriétés et méthodes nécessaires.

```
1 class User(UserMixin):
2     def __init__(self, id, username, password, role):
3         self.id = id
4         self.username = username
5         self.password = password # En pratique, stockez un
   hachage du mot de passe
6         self.role = role
7
8     # Ajoutez d'autres m thodes ou propri t s si
   n ecessaire
```

Listing 54 – Classe User

4. Chargement de l'utilisateur

La fonction `user_loader` est utilisée par Flask-Login pour charger un utilisateur à partir de son identifiant.

```
1 @login_manager.user_loader
2 def load_user(user_id):
3     # Ici, vous devez r cup rer l'utilisateur depuis votre
   base de donn es
4     # Exemple :
5     user = get_user_from_database(user_id)
6     return user
```

Listing 55 – Fonction `user_loader`

5. Routes pour l'authentification

Route pour la connexion :

```
1 from flask import request, redirect, url_for, render_template,
   flash
```

```

2
3 @app.route('/login', methods=['GET', 'POST'])
4 def login():
5     if request.method == 'POST':
6         username = request.form['username']
7         password = request.form['password']
8         user = authenticate(username, password)
9         if user:
10             login_user(user)
11             flash('Connexion réussie')
12             return redirect(url_for('index'))
13         else:
14             flash('Nom d\'utilisateur ou mot de passe incorrect')
15     return render_template('login.html')

```

Listing 56 – Route de connexion

Route pour la déconnexion :

```

1 @app.route('/logout')
2 @login_required
3 def logout():
4     logout_user()
5     flash('Vous avez terminé de connecter')
6     return redirect(url_for('login'))

```

Listing 57 – Route de déconnexion

Utilisation du décorateur login_required :

Vous pouvez protéger les routes sensibles en utilisant le décorateur @login_required :

```

1 @app.route('/control')
2 @login_required
3 def control_panel():
4     # Code pour le panneau de contrôle
5     pass

```

Listing 58 – Protection des routes

6. Gestion des rôles et des permissions

Pour implémenter le contrôle d'accès basé sur les rôles, vous pouvez créer des décorateurs personnalisés ou vérifier le rôle de l'utilisateur dans vos vues.

Exemple de décorateur pour les administrateurs :

```

1 from functools import wraps
2 from flask import abort
3
4 def admin_required(f):
5     @wraps(f)
6     def decorated_function(*args, **kwargs):
7         if current_user.role != 'admin':
8             abort(403) # Accès interdit
9         return f(*args, **kwargs)
10    return decorated_function
11
12 @app.route('/admin')

```

```

13     @login_required
14     @admin_required
15     def admin_panel():
16         # Code pour le panneau d'administration
17         pass

```

Listing 59 – Décorateur pour les administrateurs

7. Sécurisation des mots de passe

Il est crucial de ne jamais stocker les mots de passe en clair. Utilisez des fonctions de hachage sécurisées pour stocker les mots de passe des utilisateurs.

— Utilisez le module `werkzeug.security` :

```

1     from werkzeug.security import generate_password_hash,
        check_password_hash
2
3     # Pour cr er un hachage
4     hashed_password = generate_password_hash(password, method='
        sha256')
5
6     # Pour v rifier un mot de passe
7     check_password_hash(user.password, password)

```

Listing 60 – Hachage des mots de passe

8.2 Questions et directives

Pour renforcer la sécurité de votre application, suivez les étapes suivantes :

1. Mise en place de SSL/TLS :

- Générez un certificat SSL auto-signé en utilisant OpenSSL.
- Configurez votre application Flask pour utiliser le certificat et servir les pages via HTTPS.
- Vérifiez que la connexion à votre application est bien sécurisée (par exemple, en observant le cadenas dans la barre d'adresse du navigateur).

2. Implémentation de l'authentification :

- Installez et configurez Flask-Login dans votre application.
- Créez les routes pour la connexion et la déconnexion des utilisateurs.
- Protégez les routes sensibles en utilisant le décorateur `@login_required`.

3. Gestion des utilisateurs :

- Créez un système de stockage des utilisateurs (par exemple, en utilisant une base de données SQLite ou une liste pour commencer).
- Implémentez la fonctionnalité d'enregistrement des nouveaux utilisateurs (si nécessaire).
- Assurez-vous que les mots de passe sont hachés avant d'être stockés.

4. Contrôle d'accès :

- Définissez des rôles pour les utilisateurs (par exemple, `admin`, `opérateur`).
- Créez des décorateurs ou des vérifications dans les vues pour contrôler l'accès en fonction des rôles.
- Testez que les utilisateurs non autorisés ne peuvent pas accéder aux fonctionnalités restreintes.

5. Sécurité supplémentaire :

- Mettez en place des mesures pour prévenir les attaques courantes :
 - **Protection CSRF** : Utilisez des tokens CSRF pour les formulaires (Flask-WTF peut aider).
 - **Validation des entrées** : Validez et nettoyez les données fournies par l'utilisateur pour éviter les injections.
 - **Gestion des erreurs** : Ne divulguez pas d'informations sensibles dans les messages d'erreur.

8.3 Conclusion

La sécurité est un aspect fondamental dans le développement de systèmes de supervision industrielle. En mettant en place SSL/TLS, vous assurez la confidentialité et l'intégrité des communications entre les clients et le serveur. En implémentant un système d'authentification robuste et un contrôle d'accès efficace, vous protégez votre application contre les accès non autorisés et les actions malveillantes.

Il est important de rester vigilant et de suivre les meilleures pratiques en matière de sécurité, car les menaces évoluent constamment. N'hésitez pas à approfondir vos connaissances sur la sécurité des applications web et à utiliser des outils d'analyse pour identifier et corriger les vulnérabilités potentielles.

9 Tests et validation

La phase de tests et de validation est une étape cruciale dans le développement de tout système logiciel, y compris les systèmes de supervision industrielle. Elle permet de s'assurer que le système fonctionne conformément aux spécifications, qu'il est fiable, robuste, et qu'il répond aux besoins définis lors de l'analyse fonctionnelle. Les tests permettent de détecter les erreurs, de vérifier les performances, et d'améliorer la qualité globale du code.

9.1 Points de cours

9.1.1 Importance des tests

Les tests ont plusieurs objectifs essentiels :

- **Détection des erreurs** : Identifier les bugs ou les anomalies dans le code avant le déploiement.
- **Validation fonctionnelle** : Vérifier que le système répond aux exigences fonctionnelles spécifiées.
- **Robustesse** : S'assurer que le système peut gérer les cas limites et les entrées invalides sans planter.
- **Maintenance facilitée** : Les tests facilitent les modifications ultérieures du code en permettant de détecter rapidement les régressions.
- **Documentation vivante** : Les tests peuvent servir de documentation sur le comportement attendu du code.

9.1.2 Types de tests

Il existe plusieurs types de tests à différents niveaux du développement logiciel :

- **Tests unitaires** : Tests qui portent sur des composants individuels du code, tels que des fonctions ou des méthodes, pour vérifier qu'ils se comportent comme prévu.
- **Tests d'intégration** : Tests qui vérifient les interactions entre les différents modules ou composants du système.

- **Tests système** : Tests qui valident le système complet dans son environnement opérationnel.
- **Tests de régression** : Tests qui s'assurent que les modifications apportées au code n'ont pas introduit de nouveaux bugs dans des fonctionnalités précédemment opérationnelles.
- **Tests de performance** : Évaluent les performances du système en termes de temps de réponse, d'utilisation des ressources, etc.
- **Tests d'acceptation** : Réalisés par les utilisateurs finaux pour valider que le système répond à leurs besoins.

9.1.3 Tests unitaires en Python avec unittest

Le module `unittest` est le framework standard de Python pour écrire et exécuter des tests unitaires. Il est inspiré du framework JUnit pour Java et fournit une approche structurée pour définir et organiser les tests.

Structure d'un test unitaire Un test unitaire avec `unittest` suit généralement cette structure :

1. **Importation du module** : Importez le module `unittest` et le code à tester.
2. **Création d'une classe de test** : Créez une classe qui hérite de `unittest.TestCase`.
3. **Définition des méthodes de test** : Chaque méthode dont le nom commence par `test_` est un cas de test individuel.
4. **Assertions** : Utilisez les méthodes d'assertion pour vérifier les conditions attendues.
5. **Exécution des tests** : Appelez `unittest.main()` pour exécuter les tests lorsque le script est exécuté directement.

Exemple de test unitaire Supposons que vous avez une fonction `calculer_temperature_moyenne` qui calcule la température moyenne à partir d'une liste de mesures.

```

1 def calculer_temperature_moyenne(temperatures):
2     if not temperatures:
3         raise ValueError("La liste des températures ne peut pas être vide.")
4     return sum(temperatures) / len(temperatures)

```

Listing 61 – Code de la fonction à tester

Voici comment écrire des tests unitaires pour cette fonction :

```

1 import unittest
2
3 class TestCalculerTemperatureMoyenne(unittest.TestCase):
4     def test_valeurs_positives(self):
5         temperatures = [20.0, 22.5, 19.0, 21.0]
6         result = calculer_temperature_moyenne(temperatures)
7         self.assertEqual(result, 20.625)
8
9     def test_valeurs_negatives(self):
10        temperatures = [-5.0, -3.5, -4.0]
11        result = calculer_temperature_moyenne(temperatures)
12        self.assertEqual(result, -4.166666666666667)
13
14    def test_liste_vide(self):

```

```

15     temperatures = []
16     with self.assertRaises(ValueError):
17         calculer_temperature_moyenne(temperatures)
18
19     def test_valeurs_mixtes(self):
20         temperatures = [-5.0, 0.0, 5.0]
21         result = calculer_temperature_moyenne(temperatures)
22         self.assertEqual(result, 0.0)
23
24 if __name__ == '__main__':
25     unittest.main()

```

Listing 62 – Tests unitaires avec unittest

Explication du code

- `test_valeurs_positives` : Vérifie que la fonction calcule correctement la moyenne de températures positives.
- `test_valeurs_negatives` : Vérifie le calcul avec des températures négatives.
- `test_liste_vide` : Vérifie que la fonction lève une exception `ValueError` lorsque la liste est vide.
- `test_valeurs_mixtes` : Vérifie le calcul avec une combinaison de valeurs négatives et positives.
- `with self.assertRaises(ValueError)` : Vérifie qu’une exception `ValueError` est levée.

Exécution des tests Lorsque vous exécutez le script, `unittest.main()` recherche automatiquement les méthodes de test dans la classe et les exécute. Le résultat s’affiche dans le terminal, indiquant le nombre de tests exécutés et s’ils ont réussi ou échoué.

9.1.4 Autres frameworks de tests en Python

Bien que `unittest` soit le framework standard, d’autres frameworks offrent des fonctionnalités avancées et une syntaxe plus concise, tels que `pytest` et `nose2`.

Tests unitaires avec pytest `pytest` est un framework de tests puissant qui simplifie l’écriture des tests grâce à une syntaxe plus concise et à des fonctionnalités supplémentaires.

Installation de pytest :

```

1 $ pip install pytest

```

Listing 63 – Installation de pytest

Exemple de test avec pytest :

```

1 def calculer_temperature_moyenne(temperatures):
2     if not temperatures:
3         raise ValueError("La liste des temp ratures ne peut pas tre
4             vide.")
5     return sum(temperatures) / len(temperatures)
6
7 def test_valeurs_positives():
8     temperatures = [20.0, 22.5, 19.0, 21.0]
9     assert calculer_temperature_moyenne(temperatures) == 20.625

```

```

10 def test_valeurs_negatives():
11     temperatures = [-5.0, -3.5, -4.0]
12     assert calculer_temperature_moyenne(temperatures) ==
        -4.166666666666667
13
14 def test_liste_vide():
15     temperatures = []
16     with pytest.raises(ValueError):
17         calculer_temperature_moyenne(temperatures)

```

Listing 64 – Tests unitaires avec pytest

Avantages de pytest

- **Syntaxe simplifiée** : Pas besoin de classes ou de méthodes spéciales, les fonctions de test sont détectées automatiquement.
- **Rapports améliorés** : Les messages d’erreur sont plus détaillés, facilitant le débogage.
- **Extensions** : De nombreux plugins sont disponibles pour étendre les fonctionnalités (tests paramétrés, fixtures, etc.).

9.1.5 Tests d’intégration

Les tests d’intégration vérifient que les différents modules du système fonctionnent correctement ensemble. Dans le contexte de votre projet, vous pourriez tester l’intégration entre :

- Les scripts d’acquisition de données et la base de données.
- L’interface utilisateur et le backend.
- Les services ROS (Robot Operating System) entre eux.

Exemple de test d’intégration Supposons que vous avez une fonction qui lit les données du capteur et les stocke dans la base de données. Vous pouvez écrire un test qui :

1. Initialise une base de données de test.
2. Exécute la fonction d’acquisition et de stockage.
3. Vérifie que les données sont correctement insérées dans la base de données.

```

1 import unittest
2 import sqlite3
3 import os
4
5 class TestStockageDonnees(unittest.TestCase):
6     def setUp(self):
7         # Configuration avant chaque test
8         self.db_name = 'test_mesures.db'
9         self.conn = sqlite3.connect(self.db_name)
10        self.cursor = self.conn.cursor()
11        # Cr ation de la table
12        self.cursor.execute('''CREATE TABLE mesures (
13                                id INTEGER PRIMARY KEY AUTOINCREMENT,
14                                capteur TEXT,
15                                type_mesure TEXT,
16                                valeur REAL,
17                                unite TEXT,
18                                horodatage DATETIME)''')

```

```

19         self.conn.commit()
20
21     def tearDown(self):
22         # Nettoyage après chaque test
23         self.conn.close()
24         os.remove(self.db_name)
25
26     def test_insertion_donnees(self):
27         # Fonction tester
28         def inserer_donnees(capteur, type_mesure, valeur, unite):
29             horodatage = '2023-01-01 12:00:00'
30             self.cursor.execute("INSERT INTO mesures (capteur,
31                                 type_mesure, valeur, unite, horodatage) VALUES (?, ?, ?,
32                                     ?, ?)",
33                                   (capteur, type_mesure, valeur, unite,
34                                    horodatage))
35
36             self.conn.commit()
37
38         # Appel de la fonction
39         inserer_donnees('DHT22', 'Température', 25.5, 'C')
40
41         # Vérification
42         self.cursor.execute("SELECT * FROM mesures")
43         result = self.cursor.fetchall()
44         self.assertEqual(len(result), 1)
45         self.assertEqual(result[0][1], 'DHT22')
46         self.assertEqual(result[0][2], 'Température')
47         self.assertEqual(result[0][3], 25.5)
48         self.assertEqual(result[0][4], 'C')
49
50 if __name__ == '__main__':
51     unittest.main()

```

Listing 65 – Test d'intégration pour le stockage des données

Explication du code

- **setUp** : Méthode appelée avant chaque test pour configurer l'environnement (ici, création d'une base de données temporaire).
- **tearDown** : Méthode appelée après chaque test pour nettoyer l'environnement (fermeture de la connexion et suppression du fichier de base de données).
- **test_insertion_donnees** : Teste la fonction d'insertion des données en vérifiant que les données sont correctement insérées.

9.1.6 Bonnes pratiques pour les tests

- **Isolation des tests** : Chaque test doit être indépendant des autres pour éviter les effets de bord.
- **Nomination claire** : Utilisez des noms de tests descriptifs pour faciliter la compréhension.
- **Couverture de code** : Essayez de couvrir le plus de cas possibles, y compris les cas limites et les erreurs.
- **Automatisation** : Intégrez les tests dans votre processus de développement continu pour les exécuter régulièrement.

- **Documentation** : Documentez les tests pour expliquer ce qui est testé et pourquoi.

9.2 Questions et directives

Pour cette partie du projet, il est attendu que vous :

1. **Identifiez les fonctions critiques** de votre code qui nécessitent des tests, par exemple :
 - Les fonctions d’acquisition de données.
 - Les fonctions de traitement ou d’analyse des données.
 - Les interfaces avec la base de données.
 - Les services de contrôle des équipements.
2. **Écrivez des tests unitaires** pour ces fonctions en utilisant `unittest` ou `pytest`. Assurez-vous de couvrir les cas normaux, les cas limites et les cas d’erreur.
3. **Écrivez des tests d’intégration** pour vérifier que les modules communiquent correctement entre eux. Par exemple, testez que les données collectées sont bien stockées et récupérées par l’interface utilisateur.
4. **Automatisez l’exécution des tests** en utilisant un script ou un outil d’intégration continue. Cela vous permettra de détecter rapidement les régressions lors des modifications du code.
5. **Documentez vos tests** en expliquant les objectifs de chaque test, les conditions initiales, les entrées, les sorties attendues et les résultats obtenus.
6. **Analysez les résultats** : Si des tests échouent, identifiez la cause et corrigez le code si nécessaire.

9.3 Présentation des résultats des tests

Dans votre rapport, présentez les résultats des tests de manière claire :

- Incluez des extraits de code des tests les plus significatifs.
- Fournissez des captures d’écran ou des sorties du terminal montrant l’exécution des tests.
- Commentez les résultats en expliquant ce qu’ils démontrent.
- Si des problèmes ont été détectés, décrivez les corrections apportées.

9.4 Conclusion

La mise en place d’une stratégie de tests rigoureuse est indispensable pour garantir la fiabilité et la qualité de votre système de supervision industrielle. Les tests vous permettent non seulement de détecter et de corriger les erreurs, mais aussi de gagner en confiance dans le fonctionnement de votre application. En investissant du temps dans les tests, vous faciliterez la maintenance future du code et vous réduirez les risques de dysfonctionnements en production.

N’oubliez pas que les tests sont un processus itératif : ils doivent être mis à jour et enrichis au fur et à mesure que votre application évolue. Adoptez les bonnes pratiques présentées dans cette section pour améliorer continuellement la qualité de votre projet.

10 Documentation et présentation

10.1 Instructions pour le rapport LaTeX professionnel

Vous devez rédiger un rapport professionnel en utilisant LaTeX, qui inclura :

- **Page de garde** : Titre du projet, noms des étudiants, date, logo de l’institution.

- **Table des matières** : Générée automatiquement.
 - **Introduction** : Présentation du contexte et des objectifs du projet.
 - **Analyse fonctionnelle** : Reprise et approfondissement de l’analyse réalisée en début de projet.
 - **Description du système** : Détails sur le matériel utilisé, les choix logiciels, l’architecture du système.
 - **Implémentation** : Explications sur le développement, avec des extraits de code commentés.
 - **Tests et résultats** : Présentation des tests effectués et des résultats obtenus.
 - **Conclusion et perspectives** : Bilan du projet et suggestions pour des améliorations futures.
 - **Annexes** : Schémas, diagrammes, documentation technique supplémentaire.
- Conseils pour la rédaction :**
- Utilisez un style clair et professionnel.
 - Structurez votre document avec des sections et sous-sections numérotées.
 - Illustrez vos propos avec des figures, des tableaux, et des schémas.
 - Numérotez les figures et les tableaux, et faites-y référence dans le texte.
 - Citez vos sources et ajoutez une bibliographie à la fin du document.
 - Relisez attentivement pour corriger les fautes d’orthographe et de grammaire.

Conclusion

Ce projet constitue une opportunité pour développer une expertise pratique dans la conception et la mise en œuvre de systèmes de supervision industrielle. En intégrant des compétences en électronique, programmation, réseaux, sécurité, développement d’interfaces utilisateur et en utilisant ROS pour une architecture modulaire, vous serez en mesure de créer un système complet répondant aux besoins réels de l’industrie.

Le succès de ce projet repose sur une approche méthodique, une attention aux détails, et une volonté de résoudre les problèmes rencontrés. N’hésitez pas à approfondir les sujets présentés et à explorer des solutions innovantes.

Livrables attendus

- **Code source** : Scripts d’acquisition, de traitement des données, de l’interface utilisateur, et des nœuds ROS, avec commentaires.
- **Documentation** : Technique et utilisateur, sous forme de documents clairs et structurés.
- **Rapport LaTeX professionnel** : Expliquant le montage du projet, l’analyse fonctionnelle, la mise en œuvre, les tests, et les résultats.
- **Rapport de tests** : Incluant les plans de tests, les résultats, et les analyses.
- **Présentation** : Support visuel pour la présentation orale du projet.
- **Schémas et diagrammes** : Connexions matérielles, architecture logicielle, schémas de la base de données.

Ressources supplémentaires

Pour vous aider dans ce projet, voici quelques ressources utiles :

- **Documentation officielle du Raspberry Pi** : <https://www.raspberrypi.org/documentation/>
- **Tutoriels sur l’interfaçage des capteurs** : Sites tels qu’Adafruit, SparkFun, ou Instructables.

- **Documentation de Flask** : <https://flask.palletsprojects.com/>
- **Documentation de Django** : <https://www.djangoproject.com/>
- **Documentation de Tkinter** : <https://docs.python.org/3/library/tk.html>
- **Bibliothèques de visualisation** : <https://matplotlib.org/>, <https://www.chartjs.org/>, <https://d3js.org/>
- **Sécurité web** : OWASP Top Ten <https://owasp.org/www-project-top-ten/>
- **Tutoriels sur SSH et les connexions à distance** : <https://www.ssh.com/ssh/>
- **Apprendre LaTeX** : <https://www.latex-project.org/>
- **Documentation officielle de ROS** : <http://wiki.ros.org/>
- **Tutoriels ROS pour débutants** : <http://wiki.ros.org/ROS/Tutorials>